

3장. 성향점수매칭에서 딥러닝 PSM으로

□ 이 장의 질문

- 직업훈련을 받은 사람들의 1년 뒤 평균 소득이, 받지 않은 사람들보다 300만 원 높게 나왔다. 그러면 "훈련의 효과는 300만 원"이라고 보고해도 될까?
- 이 장의 답은 "안 된다"이다. 왜 안 되는지, 그러면 효과를 어떻게 재야 하는지, 그렇게 켜 숫자를 언제 믿어도 되는지를 차례로 배운다

□ 이 장을 읽는 방법

- 이 장은 두 파트로 나뉜다
 - 1부(3.1): "비슷한 사람끼리 짝지어 비교한다"는 기본 방법인 성향점수매칭을 처음부터 끝까지 한 번 해 보고, 그 결과를 믿어도 되는지 검사하는 법을 배운다
 - 2부(3.2~3.5): 그 검사에서 떨어졌을 때, 더 강한 도구(기계학습·딥러닝)로 다시 시도한다
- 수식은 쓰지 않는다. 모든 계산은 구체적인 숫자 예시로만 따라간다
- 실습 데이터는 정답을 알고 만든 연습용 데이터다
 - 훈련의 참 효과를 6.412로 심어 두었기 때문에, 우리가 계산한 값이 얼마나 빗나갔는지 채점할 수 있다
 - 현실 데이터에서는 이런 채점이 불가능하므로, 채점 없이도 결과를 믿을지 판정하는 방법까지 함께 배운다
- 실습 코드는 `practice/chapter03/code/` 에 있다. 스크립트를 실행하면 콘솔 출력 전체가 `practice/chapter03/results/<스크립트명>.log` 로 저장되고, 그림도 `results` 폴더에 저장된다. 본문의 실행 결과 발췌와 그림은 모두 이 코드가 실제로 계산한 값이다
- 용어 미리보기 — 본문에서 처음 나올 때마다 다시 풀어 설명하므로, 여기서는 눈에 익히기만 하면 된다

용어	쉬운 뜻
선택편향	처치를 받은 집단과 안 받은 집단이 처음부터 달라서, 결과 차이에 그 "원래 차이"가 섞이는 것
성향점수	그 사람의 특성으로 볼 때 처치를 받을 확률
매칭	처치받은 사람마다 특성이 비슷한 안 받은 사람을 짝지어 비교하는 방법
공통지지	두 집단의 성향점수 범위가 겹치는 구간. 겹쳐야 짝을 지을 수 있다
캘리퍼	짝의 점수 차이가 이 한도를 넘으면 짝으로 인정하지 않는 상한선
매칭률	처치받은 사람 가운데 짝을 찾은 사람의 비율
ATT	실제로 처치를 받은 사람들에게서 처치가 만든 평균 변화
표준편차	값들이 평균에서 보통 얼마나 떨어져 있는지 쟀 "흠어짐의 자"
SMD	두 집단의 평균 차이를 표준편차로 나눠, 단위가 다른 변수를 같은 눈금으로 쟀 값
신뢰구간	참값이 들어 있을 법하다고 계산한 범위
트리밍	성향점수가 극단인 사람을 분석에서 잘라내는 조치

1부. 성향점수를 추정해 매칭하고 균형으로 진단한다

□ 1부 개요

- 이 파트가 끝나면: 성향점수매칭의 세 단계(확률 추정 → 짝짓기 → 효과 계산)를 직접 실행해 첫 추정치 4.924를 얻고, 이 값을 보고해도 되는지 균형 검사로 판정할 수 있다
- 다루는 절: 3.1.1 ~ 3.1.6

□ 3.1.1 왜 그냥 비교하면 안 되는가

- 훈련을 받은 사람이 300만 원 더 번다는 사실 자체는 맞다. 문제는 그 300만 원이 훈련 "덕분"이라는 보장이 없다는 데 있다
- 훈련에 스스로 지원하는 사람은 애초에 구직 의지가 강하고 학력도 높은 편이어서, 훈련이 없었어도 더 벌었을 사람들이다

- 예를 들어 아침마다 조깅하는 사람들이 더 건강하다고 해서, 그 건강이 조깅 덕분이라고 말할 수는 없다
 - 원래 건강한 사람일수록 조깅을 시작하기 때문이다. 조깅(처치)을 스스로 선택한 사람들은 선택 이전부터 남들과 달랐고, 그 "원래 차이"가 건강 차이에 섞여 들어간다
 - 이렇게 결과 차이에 원래 차이가 섞이는 것을 **선택편향(selection bias)** 이라고 부른다
 - 관측된 300만 원 = 훈련의 효과 + 원래부터 있던 차이. 데이터를 아무리 들여다봐도 이 둘은 저절로 나뉘지 않는다
 - 같은 문제가 곳곳에 있다. 창업지원금을 받은 팀의 생존율이 높은 것은 준비된 팀일수록 신청서를 잘 쓰기 때문일 수 있고, 할인 쿠폰을 받은 고객의 구매액이 높은 것은 애초에 충성도 높은 고객에게 쿠폰을 보냈기 때문일 수 있다
 - 해결의 방향: **처치 전 특성이 비슷한 사람끼리만 비교**하면 된다
 - 특성이란 나이·소득처럼 처치 전에 이미 정해져 있던 개인의 조건을 말하며, 전문용어로는 **공변량(covariate)** 이라고 한다
 - 그런데 실제로 해 보면 문제가 생긴다. 맞춰야 할 특성이 하나가 아니라는 점이다
 - 예를 들어 각 특성을 "높음/낮음" 둘로만 거칠게 나뉘도, 특성이 5개면 조합이 $2 \times 2 \times 2 \times 2 \times 2 = 32$ 칸, 20개면 약 105만 칸이다
 - 표본이 2,000명이면 거의 모든 칸이 텅 비어서, 처치받은 사람과 같은 칸에 있는 비교 상대가 아예 없다
 - 특성이 늘수록 짝이 급격히 사라지는 이 문제를 **차원의 저주**라고 부른다
 - 이 장의 실습 데이터: 직업훈련 상황을 흉내 낸 2,000명짜리 연습 데이터
 - 훈련을 받은 사람 621명, 받지 않은 사람 1,379명. 특성은 나이·소득·교육년수·경력년수·자산 다섯 개
 - 연습용이므로 훈련의 참 효과를 6.412로 심어 두었다. 계산한 값을 6.412와 대조하면 얼마나 빗나갔는지 채점할 수 있다 — 현실에서는 불가능한 채점을 연습에서 해 보는 것이 이 장의 학습 장치다
 - 이 데이터에서 두 집단이 얼마나 다른지 재 보면, 나이 차이 0.836, 경력년수 차이 0.786이다
 - 이 숫자를 재는 자는 3.1.6에서 배우는 SMD인데, 지금은 "0이면 같고, 0.1을 넘으면 꽤 다르다"는 눈금만 알면 된다
 - 다섯 특성 모두 0.1을 크게 넘었다. 두 집단은 시작부터 전혀 다른 사람들이다
- 한 줄 정리: 두 집단은 시작부터 달랐다. 그냥 빼면 "효과 + 원래 차이"를 효과라고 보고하게 된다.**

□ 3.1.2 성향점수 — 다섯 개의 자를 하나로 합친다

- 다섯 특성을 다 맞출 수 없다면, 다섯을 하나로 합칠 수는 없을까? Rosenbaum과 Rubin(1983)의 답이 그것이다
 - 다섯 특성을 "**처치를 받을 확률**" 하나로 요약하고, 그 확률이 비슷한 사람끼리 짝지으면 된다. 이 확률이 **성향점수(propensity score)** 다
 - 확률이란 "같은 특성의 사람이 100명 있다면 그중 몇 명에게 그 일이 일어나는가"를 0과 1 사이 숫자로 적은 것이다. 성향점수 0.7은 "이런 특성의 사람 100명 중 약 70명이 훈련을 받는다"는 뜻이 된다
- 구조는 신용점수와 같다
 - 은행은 소득·직업·연체 이력을 하나하나 따지는 대신 이들을 점수 하나로 합산하고, 점수가 같으면 같은 등급으로 취급한다. 성향점수도 여러 특성을 점수 하나로 합치고, 점수가 같으면 같게 취급한다
 - 다만 신용점수의 계산 공식은 은행이 정해 둔 정답이 있지만, 성향점수의 정답 공식은 아무도 모른다. 이 차이가 뒤에서(3.1.6) 문제를 일으킨다
- 왜 확률 하나만 맞추면 다섯 특성을 맞춘 셈이 되는가? 예를 들어 성향점수가 똑같이 0.7인 사람 100명을 모았다고 하자
 - 그중 45세가 40명, 35세가 60명이다. 두 나이 모두 처치받을 확률이 0.7로 같으므로, 45세 중 약 28명, 35세 중 약 42명이 처치를 받는다
 - 처치받은 70명의 나이 구성은 28 대 42, 곧 40% 대 60%이고, 안 받은 30명도 12 대 18로 똑같이 40% 대 60%다
 - 점수가 같은 사람들 안에서는 처치를 받았든 안 받았든 나이 구성이 같아진다. 누가 받았는지가 나이와 무관한 제비뽑기처럼 정해진 셈이며, 이 성질이 다섯 특성 모두에 성립한다는 사실이 수학적으로 증명되어 있다
- 단, 이 방법이 성립하려면 조건이 두 개 필요하다
 - 첫째, 처치 배정에 영향을 준 특성이 데이터에 빠짐없이 들어 있어야 한다(**조건부 독립**). 예를 들어 담당자의 인상 평가로 대상이 정해졌는데 그 평가가 데이터에 없다면, 무엇을 해도 그 차이는 지울 수 없다. 이 가정은 데이터로 확인할 수 없으므로, 분석자가 신청 자격·선발 기준 같은 배정 절차를 문서로 확인해 따진다
 - 둘째, 두 집단의 성향점수 범위가 겹쳐야 한다(**공통지지**). 처치군은 전부 0.9 근처이고 대조군은 전부 0.2 근처라면 짝지을 상대가 없다

한 줄 정리: 다섯 개의 자를 따로 대지 말고, "**처치받을 확률**"이라는 자 하나로 켜다.

□ 3.1.3 1단계 — 확률을 계산하고, 분포부터 본다

- 성향점수는 데이터 어디에도 적혀 있지 않다. 데이터에 있는 것은 "받았다/안 받았다"뿐이고, "받을 확률이 몇이었나"는 없다
 - 그래서 컴퓨터에 계산을 시킨다. "이런 특성의 사람들은 실제로 몇 %나 받았는가"를 데이터 전체에서 학습해, 사람마다 확률 하나씩을 붙인다
 - 특성을 넣으면 값을 내주는, 데이터로 만들어 낸 계산 공식을 통계에서는 **회귀 모형**이라고 부르며, 전통적으로는 로지스틱 회귀라는 가장 단순한 직선형 공식을 쓴다
- 확률을 얻었으면 개별 값보다 먼저 **전체 분포**를 본다. 확인할 것은 세 가지다
 - 점검 1. 두 집단의 분포가 겹치는가. 겹치지 않으면 짝지을 상대가 없으므로, 매칭을 진행하지 않고 분석 대상부터 다시 정한다
 - 점검 2. 극단값(0.05 미만이거나 0.95 초과)이 몇 %인가. 이 데이터에서는 7.8%였다. 5%를 넘으면 극단 구간을 잘라내는 **트리밍(trimming)** 을 검토한다
 - 점검 3. 값이 좁은 구간에 뭉쳐 있지 않은가. 뭉쳐 있으면 공식이 처치와 무관한 특성만 쓰고 있다는 신호다
- 실습 코드(핵심 부분, 전체는 3-1-traditional-psm-limitations.py)

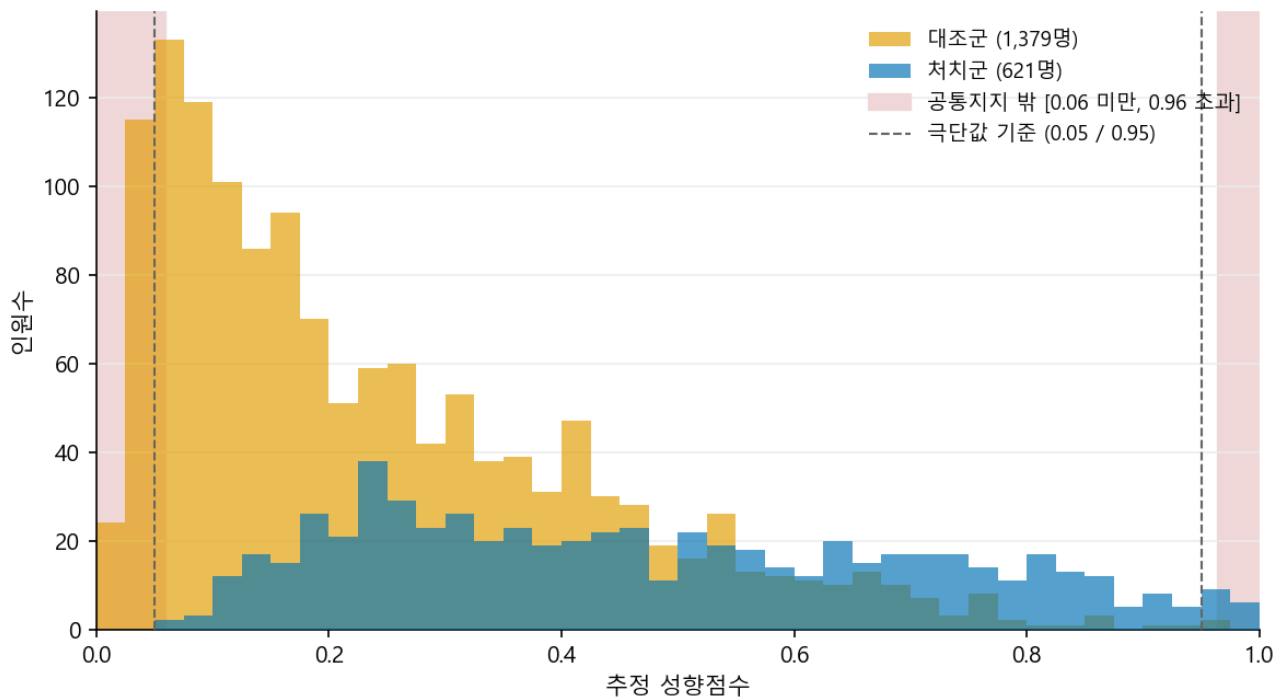
```
X = StandardScaler().fit_transform(data[feature_cols]) # 특성의 단위를 통일
model = LogisticRegression(max_iter=1000).fit(X, T) # 처치 여부를 예측하는 공식을 학습
ps = model.predict_proba(X)[: , 1] # 사람마다 처치 확률 하나
```

- 실행 결과(results/3-1-traditional-psm-limitations.log 발췌)

```
[성향점수 분포]
처치군 PS 범위: [0.059, 0.991]
대조군 PS 범위: [0.016, 0.964]

[극단 성향점수 (trimming 권장)]
PS < 0.05: 139 (7.0%)
PS > 0.95: 17 (0.8%)
```

- 해석: 두 집단의 점수 범위가 [0.059, 0.964] 구간에서 겹치므로 점검 1은 통과다. 극단값은 139 + 17 = 156명으로 전체의 7.8% — 기준 5%를 넘으므로 트리밍 검토 대상이다(점검 2)



○ 그림 3-1 읽는 법

- 파란 막대(처치군)는 오른쪽으로, 주황 막대(대조군)는 왼쪽으로 쏠려 있지만, 가운데 넓은 구간에서 겹치므로 짝짓기는 가능하다(점검 1 통과)
- 양 끝의 붉은 음영은 겹치지 않아 비교가 불가능한 구간이고, 극단값 7.8%는 주로 왼쪽 끝에 몰려 있다

○ 직접 해 보기: practice/chapter03/code 폴더에서 `python 3-1-traditional-psm-limitations.py` 실행
→ 콘솔의 "3. 공통지지 영역 분석" 부분에서 세 가지 점검 결과를 확인한다

한 줄 정리: 점수를 만들면 값보다 분포부터 본다. 분포가 안 겹치면 그다음은 없다.

□ 3.1.4 2단계 — 짝을 짓되, 억지 짝은 만들지 않는다

- 처치받은 621명 각각에게, 성향점수가 가장 가까운 대조군 한 명을 짝으로 붙인다. 이것이 **최근접 이웃 매칭**이다
- 여기에 안전장치를 하나 단다
 - 예를 들어 키 175cm인 사람의 짝을 찾는데 가장 비슷한 후보가 195cm뿐이라면, "가장 비슷하다"는 이유로 짝지어 봐야 비교가 되지 않으므로 짝 없음으로 처리하는 편이 낫다
 - 이렇게 "점수 차이가 이 한도를 넘으면 짝으로 인정하지 않는다"는 상한선이 **캘리퍼(caliper)**이며, 이 실습에서는 0.05로 두었다
 - 캘리퍼를 좁힐수록 짝의 질은 좋아지지만, 짝을 못 찾는 사람이 늘어난다

○ 이 단계의 성적은 **매칭률**, 곧 짝을 찾은 처치군의 비율로 판정한다

– 95% 이상: 그대로 진행한다

– 80~95%: 진행하되, 짝을 못 찾은 사람들이 어떤 사람들인지 확인해 보고서에 적는다

– 80% 미만: 결과를 보고하지 않고 설계를 고친다

○ 실습 코드(핵심 부분)

```
nn = NearestNeighbors(n_neighbors=1).fit(ps[T == 0].reshape(-1, 1))
distances, indices = nn.kneighbors(ps[T == 1].reshape(-1, 1))
valid = distances.flatten() <= 0.05      # 캘리퍼: 이 거리를 넘는 짝은 제외
```

○ 실행 결과(로그 발췌)

```
[매칭 결과]
처치군 전체:      621명
매칭 성공:        621명 (100.0%)
매칭 거리 중위수: 0.0003
매칭 거리 최대:   0.0265
```

– 해석: 매칭률 100%로 기준(95% 이상)을 통과했다. 가장 먼 짝의 거리도 0.0265로 캘리퍼 0.05 안이므로, 걸려진 짝 없이 전원이 가까운 짝을 찾았다. 참고로 같은 대조군 한 명이 여러 처치군의 짝으로 거듭 뿔힐 수도 있다

○ 판정 연습(가상의 수치다): 세 분석의 매칭률이 각각 97%, 88%, 62%라면?

– 97%는 그대로 진행, 88%는 미매칭자를 확인한 뒤 진행, 62%는 중단하고 캘리퍼를 넓히거나 분석 대상을 다시 정한다

한 줄 정리: 가장 가까운 짝을 찾되, 너무 먼 짝은 억지로 만들지 않는다.

□ 3.1.5 3단계 — 효과를 계산하고, 정답과 맞춰 본다

○ 짝을 다 지었으면 남은 계산은 빼기와 평균뿐이다

– 짝마다 "처치받은 사람의 소득 – 짝지어진 대조군의 소득"을 구하고, 그 값들을 평균한다. 예를 들어 짝이 세 쌍뿐이고 차이가 5, 7, 3이라면 평균은 $(5+7+3) \div 3 = 5$ 다

– 이렇게 얻은 값이 **ATT**, 곧 "실제로 처치를 받은 사람들에게서 처치가 만든 평균 변화"다

○ 평균 하나만 보고하지는 않는다

- 같은 조사를 다시 하면 값이 조금 달라질 것이므로, "참값이 들어 있을 법한 범위"도 함께 계산한다. 이것이 **95% 신뢰구간**이며, 이 범위가 0을 벗어나 있으면 "효과가 없다"고 보기는 어렵다는 뜻으로 읽는다
- 범위를 계산할 때는 **표준편차** — 값들이 평균에서 보통 얼마나 떨어져 있는지를 잴 "흩어짐의 자" — 를 쓴다

○ 실습 코드(핵심 부분)

```
diff = Y[matched_treated] - Y[matched_control] # 짝마다 결과의 차이
att = diff.mean() # 차이들의 평균 = ATT
se = diff.std(ddof=1) / np.sqrt(len(diff))
ci = (att - 1.96 * se, att + 1.96 * se) # 95% 신뢰구간
```

○ 실행 결과(로그 발췌)

```
[ATT 추정 결과]
추정 ATT:      4.924
95% 신뢰구간:  [3.579, 6.268]

[편향 분석]
진정한 ATT:      6.412
편향 비율:      -23.2%
95% CI가 진정한 값 포함: x
```

○ 실습 결과 읽기: ATT **4.924**, 95% 신뢰구간 **[3.579, 6.268]**

- 구간이 0을 벗어나므로 "훈련은 효과가 있다"고 말할 수 있다. 여기까지만 보면 성공한 분석처럼 보인다

○ 그런데 이 데이터에는 정답이 있다. 참 효과는 **6.412**였다

- 추정치 4.924는 참값보다 23% 작고, 신뢰구간 [3.579, 6.268]조차 6.412를 담지 못했다. 방향("효과가 있다")은 맞았지만 크기를 4분의 1이나 깎아 먹은 것이다
- 더 중요한 사실: 현실 데이터였다면 참값이 없으므로, 우리는 빛나간 줄도 모른 채 4.924를 보고했을 것이다
- 덧붙여 여기서 쓴 오차 계산은 연습용 단순 방식이라, 실제 불확실성보다 작게 나올 수 있다

○ 직접 해 보기: 같은 로그의 "7. 비교: Naive 추정" 부분을 확인한다. 매칭 없이 두 집단 평균을 그냥 뺀 값은 이번 실행에서 **13.449(편향 +109.7%)** — 참값 6.412의 두 배가 넘는 값을 보고할 뻔한 셈이며, 매칭(-23.2%)이 편향을 크게 줄였음을 확인할 수 있다

한 줄 정리: 방향은 맞췄지만 크기가 틀렸고, 현실이라면 틀린 줄도 몰랐을 것이다. 그래서 검사가 필요하다.

□ 3.1.6 결과를 믿어도 되는지 검사한다 — 균형 진단

- 참값 없이 무엇으로 검사할까? 매칭의 목적을 떠올리면 답이 나온다
 - 목적은 두 집단을 비슷하게 만드는 것이었다. 그렇다면 **매칭 후에 실제로 비슷해졌는지**를 재면 된다. 비슷해지지 않았다면 그 추정치는 믿을 수 없다
- 재는 자가 **SMD(표준화 평균차)** 다
 - 두 집단의 평균 차이를 표준편차로 나눈 값인데, 나누는 이유는 단위를 없애기 위해서다. 소득 차이 50만 원과 교육 차이 0.5년은 그대로는 비교할 수 없지만, 각자의 표준편차로 나누면 같은 눈금이 된다
 - 예를 들어 처치군 평균 나이가 42세, 대조군이 38세, 표준편차가 8세라면 SMD는 $(42-38) \div 8 = 0.5$ 이고, "나이 차이가 표준편차의 절반만큼"이라는 뜻이다
- 눈금은 세 단계다(Austin, 2011): **절댓값 0.1 미만 합격, 0.1~0.2 경계, 0.2 초과 불합격**. 매칭 전과 후의 SMD를 나란히 놓고 판정한다
- 실습 코드(핵심 부분)

```
pooled_std = np.sqrt((x_treat.var(ddof=1) + x_control.var(ddof=1)) / 2) # 통합 표준편차
smd = (x_treat.mean() - x_control.mean()) / pooled_std                # 특성 하나의 SMD
```

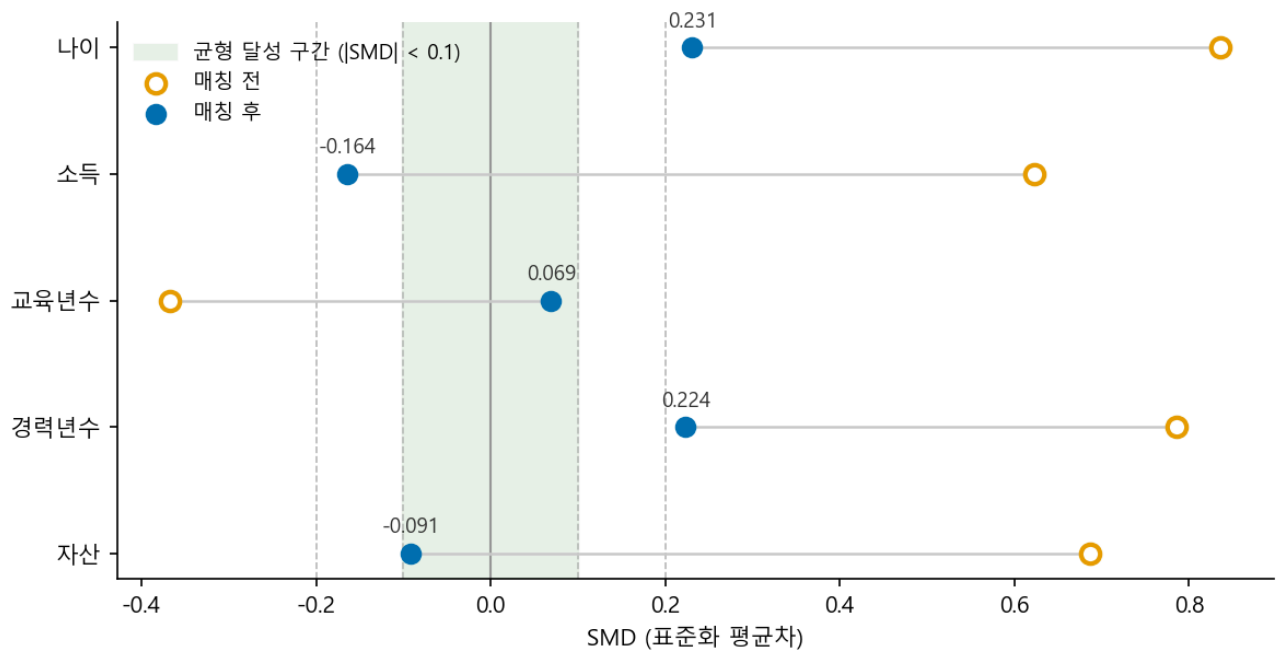
- 실행하면 콘솔 "5. 매칭 후 공변량 균형 분석"에 특성별 매칭 전후 SMD가 찍히며, 그 출력을 정리한 것이 표 3-1이다

<표 3-1: 매칭 전후 균형 검사 결과>

특성	매칭 전 SMD	매칭 후 SMD	판정
나이	0.836	0.231	불합격
소득	0.623	-0.164	경계 (부호 반전)
교육년수	-0.368	0.069	합격
경력년수	0.786	0.224	불합격
자산	0.687	-0.091	합격
평균	0.660	0.156	불합격 (기준 0.1)

○ 표 읽는 요령 두 가지

- 첫째, **개선율이 아니라 최종 수준으로 판정한다**. 평균 SMD가 0.660에서 0.156으로 76% 줄었지만, 기준은 0.1이고 최종값은 0.156이므로 불합격이다. "많이 좋아졌다"와 "충분히 비슷해졌다"는 다른 말이다
- 둘째, **부호가 뒤집힌 특성을 주의한다**. 소득은 +0.623에서 -0.164가 되었다. 처치군보다 소득이 높은 대조군을 지나치게 골라 반대 방향으로 넘어간 것으로, 과잉 보정의 신호다



○ 그림 3-2 읽는 법: 주황 빈 점(매칭 전)이 파란 점(매칭 후)으로 이동한 거리가 개선 폭이고, 초록 띠 (± 0.1) 안에 들어온 파란 점만 합격이다. 다섯 중 둘만 띠 안에 있고, 소득의 파란 점은 0을 지나쳐 반대편에 가 있다

○ 불합격이면 할 수 있는 조치는 세 가지다

- ① 매칭 조건을 바꾼다(캘리퍼 조정 등) ② 성향점수를 만드는 공식 자체를 바꾼다 ③ 남은 차이를 통계적으로 추가 보정한다
 - 이 실습은 ②를 택한다. 원인이 공식에 있기 때문이다. 이 데이터의 처치 배정 규칙은 곡선 모양인데 로지스틱 회귀는 직선형 공식이라 그 곡선을 따라가지 못했고, 추정한 점수와 참 점수의 닮은 정도 (상관)는 0.665로 절반 남짓밖에 안 닮았다. 공식이 틀린 점수를 낸 것이며, 이를 **모델 오설정**이라고 한다
 - 덧붙여 이 데이터는 표본의 9.9%가 분포가 겹치지 않는 구간에 있었으므로, 추정한 효과는 표본 전체가 아니라 겹치는 구간의 사람들에게만 해당한다는 단서도 보고서에 적어야 한다
- 판정 연습(가상의 수치다): 어떤 매칭의 결과가 SMD 0.08, -0.12, 0.05, 0.19, 0.03이라면?
- 절댓값 평균은 0.094로 기준을 간신히 통과하지만, 0.19와 -0.12 두 특성이 경계 구간에 있다. 그대로 보고하지 않고, 추가 보정을 하거나 매칭을 다시 한 뒤 재검사한다

한 줄 정리: 76% 좋아졌어도 0.156은 불합격이다. 판정은 개선율이 아니라 최종 수준으로 한다.

□ 1부 정리

- 지금까지 만든 것 네 가지: 사람마다 붙인 성향점수, 621쌍의 짝, 효과 추정치 4.924와 신뢰구간, 균형 검사 결과
- 확인한 판정 기준
 - 극단 점수(0.05 미만·0.95 초과)가 5%를 넘으면 트리밍을 검토한다
 - 매칭률이 80% 미만이면 보고하지 않는다
 - SMD는 평균·최대 모두 절댓값 0.1 미만이어야 합격이고, 부호 반전은 과잉 보정 신호다
- 결론: 균형 검사 불합격 → ATT 4.924는 보고하지 않는다. 직선 공식이 못 잡은 곡선을 잡을 도구가 필요하며, 그것이 2부의 주제다
- 자가 점검(가상의 수치다)
 - 평균 SMD 0.07·최대 0.09·매칭률 96%라면? 모두 합격이므로 보고한다
 - 평균 0.09·최대 0.24·매칭률 98%라면? 최대가 0.2를 넘으므로 보고하지 않고 설계를 고친다

2부. 기계학습과 딥러닝으로 성향점수를 다시 추정한다

□ 2부 개요

- 이 파트가 끝나면: 곡선을 다룰 수 있는 두 가지 방법(랜덤 포레스트·그래디언트 부스팅)으로 성향점수를 다시 만들고, 로지스틱 회귀까지 세 방법을 같은 시험대에 올려 비교·선택할 수 있다
- 다루는 절: 3.2, 3.4 ~ 3.5

□ 3.2 곡선은 컴퓨터가 스스로 찾게 한다

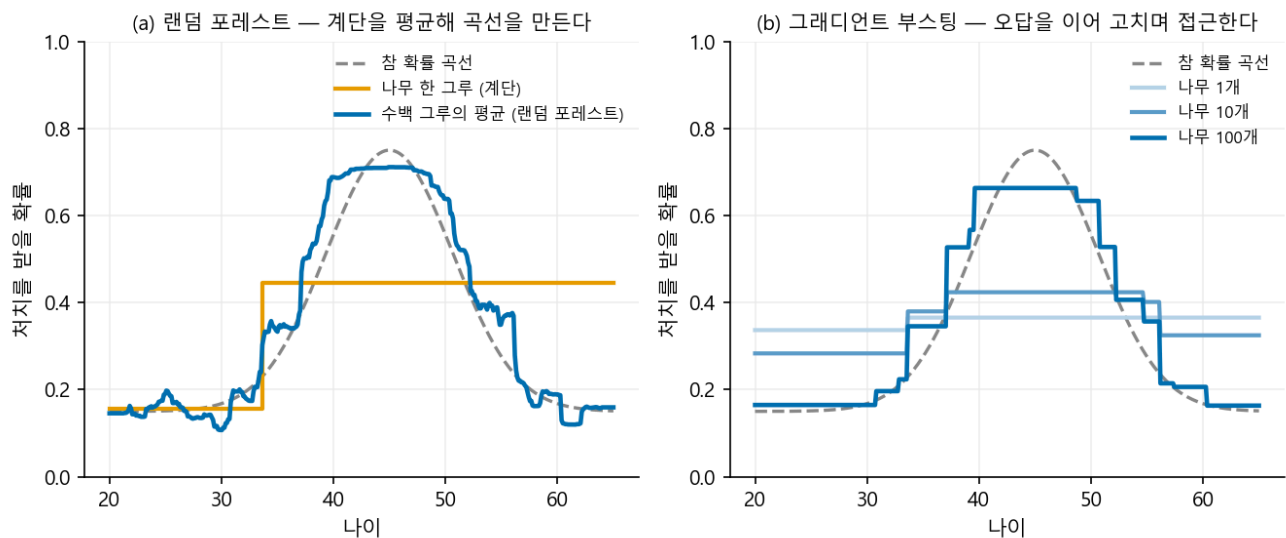
- 1부의 실패 원인은 직선 자료 곡선을 재려 한 것이었다. 그렇다고 우리가 곡선의 모양을 아는 것도 아니다
 - 어떤 항을 추가해야 할지 모른다면, **곡선의 모양을 데이터가 스스로 찾게** 하면 된다. 입력(특성)과 출력(확률)은 그대로 두고,中间的 공식만 더 유연한 것으로 바꾼다

○ 랜덤 포레스트(Random Forest): "스무고개 나무" 수백 그루의 평균이다(Breiman, 2001)

- 나무 한 그루는 "나이가 42세 이상인가?" 같은 질문으로 데이터를 계속 쪼개고, 쪼개진 칸마다 다른 확률을 준다. 예를 들어 한 번만 쪼갰 나무는 42세 이상 구간에 0.8, 미만 구간에 0.3을 주는 두 단짜리 계단이다
- 질문이 늘고 수백 그루의 답을 평균하면 계단이 촘촘해져 곡선에 가까워지고, 한 그루의 실수는 평균에 묻힌다

○ 그래디언트 부스팅(Gradient Boosting): 오답노트 방식이다(Friedman, 2001)

- 예를 들어 실제로 처치를 받은 사람에게 첫 나무가 확률 0.6을 주면 0.4만큼 틀린 것이고, 두 번째 나무는 그 틀린 만큼을 고치는 데 집중한다
- 앞 나무의 오답을 뒤 나무가 이어서 고치며 쌓아 가는 구조이며, 한 번에 고치는 양(학습률)을 작게 잡을수록 천천히 정교하게 맞춘다



○ 그림 3-5 읽는 법 — 개념 확인용으로 만든 작은 예시(나이 하나에 따라 처치 확률이 곡선을 그리는 상황)에 두 방법을 실제로 학습시킨 결과다

- 왼쪽: 나무 한 그루(주황)는 두 단짜리 계단에 그치지만, 수백 그루의 평균(파랑)은 회색 점선(참 곡선)을 거의 따라간다
- 오른쪽: 부스팅은 나무 1개일 때 거의 평평하던 선이, 10개·100개로 쌓이면서 점점 곡선에 다가간다

○ 두 방법에는 공통 함정이 있다

- 처치 여부를 너무 정확히 맞혀 버리면 — 처치군은 전부 0.99, 대조군은 전부 0.01 — 두 분포가 갈라져 짝지을 상대가 사라진다
- 그래서 나무의 깊이 같은 설정을 일부러 보수적으로 잡고, 설정을 고를 때의 채점 기준도 "얼마나 잘 맞히는가"가 아니라 "매칭 후 균형이 얼마나 좋은가"로 삼는다(McCaffrey et al., 2004)

○ 실습 코드(핵심 부분, 전체는 3-2-ml-psm.py)

```
# 랜덤 포레스트: 과적합을 막으려고 나무를 얇게(깊이 3), 잎을 크게(150명) 제한
rf = RandomForestClassifier(n_estimators=100, max_depth=3,
                           min_samples_leaf=150, random_state=42).fit(X, T)
ps_rf = rf.predict_proba(X)[:, 1]

# 그래디언트 부스팅: 반복 횟수·학습률을 "매칭 후 균형"이 가장 좋아지는 조합으로 선택
gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.01,
                                max_depth=3, random_state=42).fit(X, T)
ps_gb = gb.predict_proba(X)[:, 1]
```

○ 실행 결과(results/3-2-ml-psm.log 발췌)

```
=== Random Forest PSM ===
매칭 성공: 576쌍 (매칭률: 100.0%)
Random Forest ATT: 5.068 (SE: 0.201)
평균 SMD (매칭 후): 0.0858

=== Gradient Boosting PSM (균형-정확도 동시 최적화) ===
최적 하이퍼파라미터 (균형-정확도 Joint): n_estimators=100, learning_rate=0.01
Gradient Boosting ATT: 4.862 (SE: 0.204)
평균 SMD (매칭 후): 0.0389
```

○ 이 출력에 로지스틱 회귀(같은 데이터를 쓰는 3-1-traditional-psm.py 의 결과)를 나란히 놓은 것이 표 3-2다. 데이터는 참 효과 5.0을 심은 별도의 연습 데이터다

<표 3-2: 세 방법의 비교 (참 효과 5.0)>

방법	평균 SMD	최대 SMD	ATT
로지스틱 회귀	0.071	0.163	5.065
랜덤 포레스트	0.086	0.162	5.068
그래디언트 부스팅	0.039	0.081	4.862

○ 표 읽는 법

- 균형만 보면 그래디언트 부스팅이 유일하게 평균·최대 모두 합격이다. 로지스틱과 랜덤 포레스트는 평균은 통과했지만 최대가 걸렸다 — 여기서도 "평균만 보고 판정하지 않는다"는 규칙이 적용된다
- ATT는 세 방법 모두 참값 5.0 근처(빛나감 -2.8%~+1.4%)여서, 이 데이터에서는 어느 방법을 골라도 결론이 크게 다르지 않다

○ 직접 해 보기: `python 3-2-ml-psm.py` 실행(수십 초) → 콘솔에서 위 결과가 재현되는지 확인한다

한 줄 정리: 곡선은 기계가 찾는다. 단, 잘 찾았는지는 예측 점수가 아니라 균형으로 채점한다.

□ 3.4 균형을 학습 목표에 넣고, 그 배점을 정한다 — λ

○ 지금까지의 방법은 "처치를 잘 맞히도록" 학습시키고, 균형은 나중에 검사했다. 학습 목표와 검사 기준이 서로 다른 셈이다. 순서를 뒤집어, 균형이 어긋나면 감점하는 항목을 학습 목표에 직접 넣는 방법도 있다

– 신경망(계산 단위를 여러 층으로 쌓아 복잡한 관계를 배우는 예측 모형)은 총 감점을 줄이는 방향으로만 공부하는 학생이고, 무엇을 얼마나 감점할지 적은 표(손실 함수)가 채점 기준이다. 이 감점표에 "답 틀림" 항목과 "균형 어긋남" 항목을 함께 넣는 것이다

○ 두 감점 항목은 재는 대상이 다르다

– **예측 감점("답 틀림")**: 각 사람에게 준 확률이 실제 처치 여부와 어긋난 정도다. 실제로 받은 사람에게 0.9를 주면 감점이 작고 0.3을 주면 크며, 안 받은 사람에게 0.8을 주면 크다. 이 감점을 전원에 대해 평균한 값이고, 낮을수록 예측이 정확하다

– **균형 감점("균형 어긋남")**: 그 확률로 가중해 비교했을 때 처치군과 대조군의 특성(나이·소득 등) 평균이 어긋난 정도다. 낮을수록 두 집단의 비교가 공정하다

– 차이를 예를 들어 보면, 실제 처치율이 70%인 집단이 있다고 하자. 전원에게 0.7을 주면 개인은 아무도 정확히 못 맞혀(실제 값은 받음/안 받음뿐) 예측 감점이 있지만, "70%가 받는 집단에 0.7"은 집단 비교의 저울로는 정확해 균형 감점이 작다. 반대로 받을 사람에게 0.99, 안 받을 사람에게 0.01을 주면 개인은 거의 다 맞혀 예측 감점이 최소지만, 두 집단의 점수가 양 끝으로 갈라져 균형 감점이 커진다

– 이렇게 두 감점은 어느 정도까지 서로 반대 방향으로 당긴다(3.2의 공통 함정과 같은 긴장). 그래서 두 항목을 함께 넣되, 비중을 λ 로 조절한다

○ 그러면 균형 항목의 배점(기호로 λ)을 얼마로 할 것인가 — 배점을 정하는 일 자체가 분석자의 의사결정이다

– 배점이 0이면 학생은 균형 항목을 무시하고 예측만 공부하니 기존 방식으로 되돌아가고, 배점이 지나치게 크면 예측을 통째로 포기한다. 적절한 지점은 다이얼을 돌려 가며 찾는 수밖에 없다

○ 균형 항목의 감점은 이렇게 쟀다

– 처치군과 "처치를 받을 뻔했던 대조군"을 비교해야 공정하므로, 그런 대조군일수록 여러 명 묶으로 세어 준다

- 예를 들어 여론조사에서 20대 응답자가 적게 잡히면 20대 한 명의 답을 여러 명 몫으로 세어 인구 구성을 맞추는데, 그것과 같은 원리다. 실습에서는 성향점수 0.75인 대조군을 3명 몫으로, 0.2인 대조군을 0.25명 몫으로 센다
- 배점 λ 는 학습으로 저절로 정해지는 값이 아니라, **사람이 학습 시작 전에 정해 줘야 하는 설정값**이다. 어떤 값이 좋은지 미리 알 수 없으므로 후보를 정해 하나씩 시험한다
- "돌려 본다"의 실제 의미: 후보 여섯 개(0, 0.1, 0.3, 0.5, 0.7, 1.0)에 대해, $\lambda=0$ 으로 한 번 학습하고, $\lambda=0.1$ 로 처음부터 다시 한 번 학습하고, ... $\lambda=1.0$ 으로 또 한 번 — **모형을 여섯 번 따로 만들어** 성적을 비교한다는 뜻이다
- 예를 들어 찌개의 소금 양을 모를 때 0스푼·반 스푼·한 스푼으로 세 번 따로 끓여 맛을 비교하는 것과 같은 구조다. 소금 양이 λ 에, 한 번 끓이기가 한 번의 학습에, 맛 평가가 아래 표의 지표에 대응한다
- 실습 코드(핵심 부분, 전체는 3-4-covariate-balance.py)

```
for lam in [0.0, 0.1, 0.3, 0.5, 0.7, 1.0]:           # 배점 후보마다
    model, bce, weighted_smd, ps = train_model(X, T, lambda_balance=lam) # 처음부터 다시 학습
    att, matched_smd, n_pairs = match_and_estimate(ps, T, y, X)         # 매칭해 성적 평가
```

- 실행 결과(results/3-4-covariate-balance.log — 여섯 번 학습의 성적 전체)

Lambda	BCE	가중 SMD	매칭후 SMD	ATT	편향	매칭쌍
0.0	0.5024	0.0222	0.1008	3.822	-1.178	556
0.1	0.5381	0.0171	0.0592	4.676	-0.324	553
0.3	0.5493	0.0249	0.0595	4.528	-0.472	559
0.5	0.5686	0.0216	0.0609	4.658	-0.342	560
0.7	0.5783	0.0139	0.0776	4.599	-0.401	568
1.0	0.5583	0.0213	0.0449	4.684	-0.316	565

가중 SMD 최소: $\lambda = 0.7$ / ATT 편향 최소: $\lambda = 1.0$

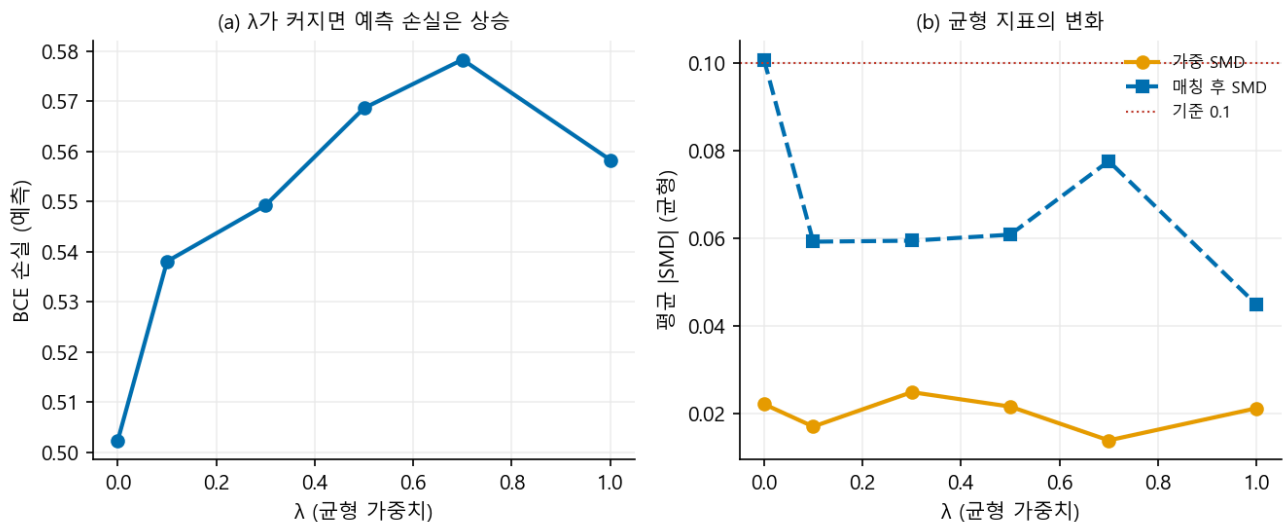
- 이 여섯 행 가운데 세 행($\lambda = 0, 0.1, 1.0$)을 추린 것이 표 3-3이다. 각 행이 따로 학습한 모형 하나의 성적표다

<표 3-3: 배점(λ)에 따른 변화 (참 효과 5.0)>

λ (배점)	예측 감점	매칭 후 SMD	효과의 빛나감
0 (균형 무시)	0.502	0.101	-1.178
0.1	0.538	0.059	-0.324
1.0	0.558	0.045	-0.316

○ 표 읽는 법

- 배점이 0일 때는 매칭 후 SMD가 0.101로 기준에 걸리고 효과 추정도 -1.178이나 빛나간다
- 배점을 0.1만 줘도 SMD가 0.059로 내려오고 빛나감도 -0.324로 줄어들며, 그 대가로 예측 감점이 0.502에서 조금씩 늘어난다. **예측을 조금 내주고 균형과 정확도를 사는 교환**이다
- 다만 균형이 가장 좋은 배점과 빛나감이 가장 작은 배점이 서로 다르고 값이 순서대로 움직이지도 않으므로, 다이얼은 촘촘히 돌려 보고 결과(ATT)가 크게 흔들리지 않는 구간에서 고른다



- 그림 3-3 읽는 법: 왼쪽은 배점을 키울수록 예측 성적이 조금씩 나빠지는 모습, 오른쪽은 그 대신 매칭 후 SMD(파란 사각)가 기준선 0.1 아래로 내려오는 모습이다

한 줄 정리: 예측을 조금 내주고 균형을 산다. 다이얼은 결과가 흔들리지 않는 구간에서 고른다.

□ 3.5 마지막 시험 — 곡선이 심할 때 어떤 방법이 살아남는가

- 지금까지의 데이터에서는 세 방법의 답이 비슷했다(빛나감 -2.8%~+1.4%). 그러면 방법 선택은 별로 중요하지 않은 것일까?
- 확인을 위해 배정 규칙을 일부러 심하게 휘어 놓은 데이터(3,000명, 참 효과 6.0)를 만들어, 세 방법을 설정 그대로 적용해 보았다

- "곡선이 심하다"는 변수가 많다는 뜻이 아니다. 이 데이터의 변수는 7개뿐이며, 그 변수들과 처치 확률 사이 **관계의 모양**이 직선에서 크게 벗어나 있다는 뜻이다
- 이 데이터에 심어 둔 휘어짐은 네 종류다: ① 제곱항(역U자 — 중간값 근처에서 확률이 가장 높고 양쪽 끝에서 낮아짐) ② 범프(특정 구간에서만 확률이 불쑥 높아짐) ③ 상호작용(한 변수의 효과가 다른 변수의 값에 따라 달라짐) ④ 주기함수(확률이 오르내림을 반복)
- 로지스틱 회귀는 "이 변수가 클수록 확률이 높다(또는 낮다)"는 한 방향의 직선만 표현할 수 있어, 이 네 가지 모양을 하나도 그리지 못한다
- 이 장에 나온 두 가지 어려움을 구분해 둔다 (표 3-4)

<표 3-4: 두 가지 어려움 — 차원의 저주와 곡선>

	차원의 저주	곡선(비선형)
문제의 정체	변수가 많아서 모든 변수가 비슷한 짝이 사라짐	변수가 적어도, 확률과의 관계 모양 이 직선이 아님
어디서 다뤘나	3.1.1	3.1.6, 3.5
해결책	성향점수로 숫자 하나로 압축	모양을 스스로 찾는 모형(랜덤 포레스트·그래디언트 부스팅)

- 현실에서는 두 문제가 함께 오는 경우가 많다. 변수가 수십 개로 늘면 상호작용 조합이 폭증해 곡선 문제도 같이 심해지며, 이것이 랜덤 포레스트류가 실무에서 선호되는 이유다
- 다만 이 실습의 요점은 "변수 7개뿐이어도 관계의 모양이 휘면 직선 공식은 크게 실패한다"는 것이다
- 실습 코드(핵심 부분, 전체는 3-5-figure-3methods.py)

```
ps_lr = LogisticRegression(max_iter=1000).fit(X, T).predict_proba(X)[:, 1]
ps_rf = RandomForestClassifier(n_estimators=200, max_depth=10,
                              min_samples_leaf=20, random_state=42).fit(X, T).predict_proba(X)[:, 1]
ps_gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1,
                                   max_depth=5, random_state=42).fit(X, T).predict_proba(X)[:, 1]

# 세 점수 각각으로 캘리퍼 0.01 매칭 → ATT·SMD를 같은 기준으로 평가
att, se, avg_smd, max_smd = match_and_evaluate(ps, T, y, X, data)
```

- 실행 결과(results/3-5-figure-3methods.log , 표기 정리)

참 ATT (True ATT): 6.000

로지스틱: ATT=11.063 SE=0.231 평균SMD=0.066 최대SMD=0.094

랜덤 포레스트: ATT=5.705 SE=0.113 평균SMD=0.075 최대SMD=0.132

그래디언트 부스팅: ATT=4.737 SE=0.129 평균SMD=0.182 최대SMD=0.364

○ 이 출력을 정리한 것이 표 3-5다

<표 3-5: 강한 곡선 데이터에서의 최종 결과 (참 효과 6.0)>

지표	로지스틱	랜덤 포레스트	그래디언트 부스팅
평균 SMD	0.066	0.075	0.182
최대 SMD	0.094	0.132	0.364
ATT	11.063	5.705	4.737
빗나감	+84.4%	-4.9%	-21.1%
학습 시간	0.01초	0.8초	1.3초

○ 가장 먼저 볼 행은 로지스틱이다

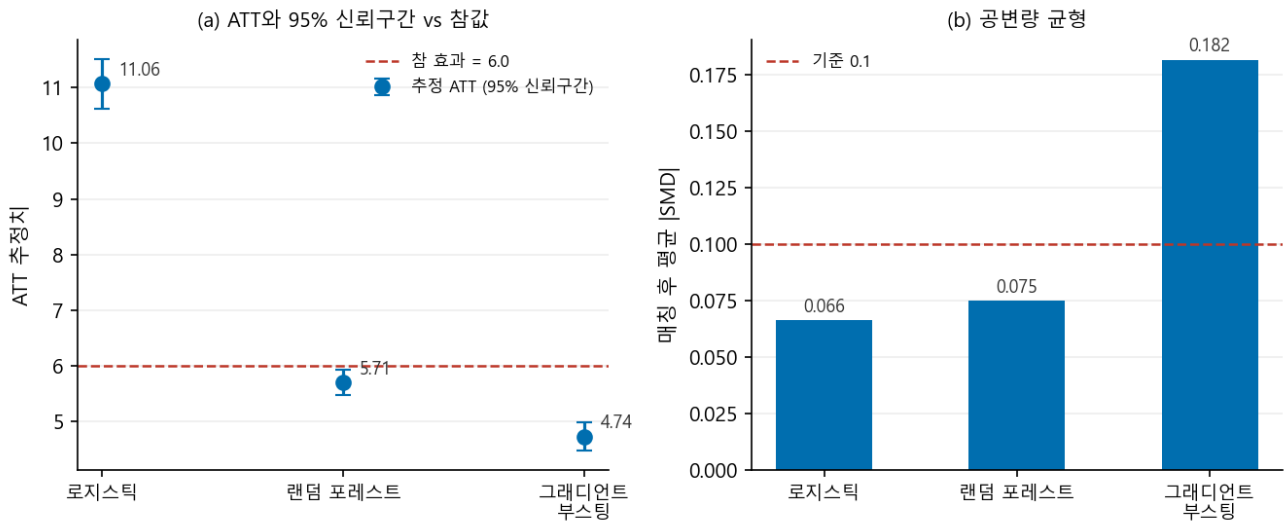
- 균형만 보면 유일하게 평균·최대 모두 합격한 1위인데, 정작 답은 11.063으로 참값 6.0보다 84%나 크다
- 원인: 짝짓기에 쓴 점수 자체가 틀렸다. 이 데이터에서 로지스틱의 점수는 참 점수와의 닮은 정도가 0.4338로 절반도 안 닮았다. 틀린 점수로 지은 짝들이, 우리가 확인하는 다섯 특성의 평균만 우연히 비슷했을 뿐, 곡선이 만든 진짜 차이는 그대로 남아 있었다

○ 건강검진과 구조가 같다

- 검진에서 쯤 다섯 항목이 모두 정상이어도, 검사하지 않은 병이 있으면 건강하다고 결론지을 수 없다. 검진 항목이 SMD로 쯤 다섯 특성이고, 검사하지 못한 병이 곡선 구조의 차이다
- **균형 합격은 필요조건이지 충분조건이 아니다**

○ 나머지 방법의 결과

- 랜덤 포레스트가 -4.9%로 가장 정확했고, 그래디언트 부스팅은 -21.1%에 균형까지 무너졌다(최대 SMD 0.364)
- 3.2 데이터에서 균형 1위였던 그래디언트 부스팅이 여기서는 균형 꼴찌가 되었다. **한 데이터에서 좋았던 방법이 다른 데이터에서도 좋으리라는 보장은 없다**
- 그리고 세 방법 모두, 신뢰구간이 참값 6.0을 담지 못했다



○ 그림 3-6 읽는 법

- 왼쪽은 방법별 추정치와 오차 범위인데, 어느 막대도 참값 6.0의 수평선에 닿지 않는다
- 오른쪽에서 균형 기준선 0.1을 통과한 방법(로지스틱·랜덤 포레스트)과, 왼쪽에서 참값에 가장 가까운 방법(랜덤 포레스트)이 일치하지 않는다. 두 그림을 대조해서 읽는 것이 요령이다

○ 참값이 없는 현실에서 이런 실패를 잡아내는 세 가지 확인

- ① 서로 다른 방법으로 만든 점수끼리 닮았는지 본다. 닮은 정도가 0.6 미만이면 적어도 하나는 틀렸다는 신호다
- ② **방법별 ATT를 나란히 놓는다.** 11.06과 4.74가 같이 나오면 어느 하나를 골라 보고할 수 없다 — 이것이 가장 중요한 습관이다
- ③ 특성과 처치의 관계를 그림으로 그려 곡선이 눈에 보이는지 확인한다

○ 실무 권장 (표 3-6)

<표 3-6: 상황별 권장>

상황	권장
곡선(비선형) 의심 — 대부분의 실제 데이터	랜덤 포레스트 (이 실험에서 빛나감 최소)
직선 관계를 확신할 근거가 있을 때	로지스틱 회귀 (가장 빠르고 해석이 쉬움)
언제나	방법 2~3개를 병행해 값이 비슷한지 확인

- 직접 해 보기: `python 3-5-figure-3methods.py` 실행(수 초) → 콘솔에 찍히는 세 방법의 ATT-SMD가 표 3-5와 일치하는지, 균형을 통과한 방법과 빛나감이 가장 작은 방법이 서로 다른지 확인한다

한 줄 정리: 균형 합격은 건강 진단서가 아니다. 방법을 바꿔도 값이 비슷한지까지 봐야 결론이 선다.

□ 결과를 읽는 순서

- PSM 분석의 산출물은 다음 순서로 읽는다. 순서를 지키지 않으면 3.5의 로지스틱처럼, 균형만 보고 틀린 값을 채택하게 된다
- 1순위 **겹침과 매칭률**: 분포가 안 겹치거나 매칭률이 80% 미만이면, 뒤의 숫자는 읽지 않고 설계부터 고친다
- 2순위 **균형(SMD)**: 평균과 최대를 모두 0.1과 대조하고, 어느 특성이 걸렸는지 확인한다
- 3순위 **점수의 타당성**: 방법을 바꿔 봐도 점수와 ATT가 비슷한지 본다. 3.5의 로지스틱은 이 단계에서 걸려야 했다
- 4순위 **효과와 신뢰구간**: 앞의 세 관문을 통과한 뒤에야 효과 크기를 읽는다. 구간이 0을 벗어나는지, 폭이 지나치게 넓지 않은지를 본다
- 5순위 **민감도**: 두세 방법의 값을 나란히 보고하고, 크게 다르면 그 사실 자체를 결과에 적는다

□ 실무 적용 점검표

- 처치 배정에 영향을 준 특성이 데이터에 다 있는가? 담당자의 재량이 기준이었다면 이 방법으로는 해결되지 않는다
- 처치 "이후"에 측정된 변수를 특성에 넣지 않았는가? 처치의 결과를 통제하면 효과의 일부를 지운다
- 두 집단의 성향점수 분포가 겹치는가?
- 표본과 매칭 쌍 수가 충분한가?
- 균형 검사(평균·최대 SMD 0.1)를 통과했는가?
- 방법을 바꿔서도 값을 확인했는가?
- 결론의 적용 범위(처치받은 사람들, 분포가 겹치는 구간)를 적었는가?
- 관측하지 못한 특성이 남아 있을 가능성을 한계로 적었는가?

□ 핵심 정리

방법	원리	유의할 점
로지스틱 회귀	직선형 공식으로 확률 계산	곡선 배정 규칙을 따라가지 못함
랜덤 포레스트	스무고개 나무 수백 그루의 평균	균형은 간접 관리
그래디언트 부스팅	오답노트식으로 오차를 이어서 보정	설정에 따라 결과가 크게 출렁임
균형 패널티 학습(λ)	감점표에 균형 항목을 추가해 학습	배점 λ 를 격자로 탐색해야 함

○ 이 장의 판정 규칙 세 가지

- **SMD 0.1 기준은 평균과 최대 모두에 적용한다.** 평균이 통과해도 최대가 걸리면 불합격이다
- **균형 합격은 필요조건이지 충분조건이 아니다.** 3.5의 로지스틱은 균형 1위였지만 효과를 84% 크게 보고할 뻔했다
- **한 데이터의 우열을 다른 데이터로 옮기지 않는다.** 방법 선택은 매번 그 데이터에서 다시 검사한다